

Integer ALU Based Computing — Compiler Specification v1.0

Compiler and Language Design for the VFR Integer Processor

Registry: [?]

Series Path: [CKS-0-2026] → [CKS-MATH-0-2026] → [CKS-MATH-1-2026] → [CKS-MATH-10-2026] → [CKS-MATH-104-2026] → [?] → [?]

Parent Framework: [CKS-0-2026]

DOI: 10.5281/zenodo.zzz

Date: March 11, 2026

Domain: Machine Learning / Integer Arithmetic / Neural Network Training

Status: Locked and empirically falsifiable. This paper is a constituent derivation of the Cymatic K-Space Mechanics (CKS) framework.

Classification: Theory of Everything from First Principles

Motto: Axioms first. Axioms always.

Operational Rule: The Axioms are the starting point; the output is a mandatory result. Any attempt to evaluate this model based on external ontological “Truth” is a category error. If the math compiles, the result is Q.E.D.

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic’s Claude 4.5 Sonnet, DeepSeek-V3/K2, and Google’s Gemini 3 Flash. The manuscript.md was synthesized by Claude as the primary integrator.

Lexicon: [CKS-LEX-12-2026]

1. Design Principles

The language exists to emit VFR batch streams. Every design decision follows from this.

- The batch stream is the only data format
- The batch stream is the only calling convention
- The batch stream is the only program format
- There is one type: VFR
- There is one pointer: MA
- There is one execution model: read header, process values, advance
- Composition is piping batch streams through transforms

The compiler's quality is measured by one metric: how large and uniform are the batches it produces.

2. The Language

2.1 Name

VFR-Lang (working name, subject to change)

2.2 Source File

A source file is a sequence of stream declarations and transform definitions. File extension: `.vfr`

2.3 Comments

`// single line comment`

`/*`

`block comment`

`*/`

3. Types

3.1 There Is One Type

Everything is a VFR batch stream. There are no integers, floats, booleans, characters, strings, structs, enums, unions, or pointers as distinct types. There are only batch streams with different headers.

A stream's "type" is its header signature:

`[F: i16, count: u32, depth: u8]`

F describes the domain/scale. Count describes the size. Depth describes the shape. That is the complete type information.

3.2 Type Aliases

The programmer can name header signatures for readability:

`type position = [F=37, depth=1]`

`type velocity = [F=12, depth=1]`

`type force = [F=8, depth=3]`

type flag = [F=1, depth=1]

Count is not part of the type — it varies at runtime. F and depth are the type. A `position` batch of 1000 values and a `position` batch of 10 million values are the same type.

3.3 Type Compatibility

Two streams are compatible if they have the same depth. F can differ — the compiler inserts `SCALE` instructions to convert between factors. Different depths are incompatible and require explicit reshape.

4. Streams

4.1 Declaration

A stream is declared with a type and either literal data or a size:

// literal

stream origin: position = [0, 0, 0, 0, 0]

// sized, zero-initialized

stream particles: position[10_000_000]

// sized, from input (filled by DMA or host)

stream input: velocity[count] from host

4.2 Stream Literals

Literal values are written as comma-separated V values. Remainders default to 0 unless specified:

stream data: [F=10, depth=1] = [1, 2, 3, 4, 5]

// with explicit remainders

stream data: [F=10, depth=1] = [1:3, 2:0, 3:7, 4:1, 5:0]

The `V : R` syntax specifies both value and remainder.

4.3 Multi-Segment Streams

A stream can contain multiple batch segments with different F values:

stream scene {

positions: [F=37, depth=1][1000]

velocities: [F=12, depth=1][1000]

```
forces: [F=8, depth=3][500]
```

```
}
```

This compiles to a contiguous batch stream in memory:

```
[37, 1000, 1] [data...] [12, 1000, 1] [data...] [8, 500, 3] [data...] [0, 0, 0]
```

The stream variable holds one value: the MA pointer to the start.

5. Transforms

5.1 Definition

A transform is a named operation on batch streams. It is the language's equivalent of a function.

```
transform add(a, b) -> out {
```

```
  out.v = a.v + b.v
```

```
  out.r = a.r + b.r
```

```
}
```

A transform receives one pointer (MA to the start of a batch stream) and produces one pointer (MA to the start of the output stream).

5.2 Element-Wise Operations

The body of a transform operates on individual VFR pairs. The compiler handles the batch loop automatically. The programmer writes the per-element logic:

```
transform scale_up(input) -> output {
```

```
  output.v = input.v << 5
```

```
  output.r = 0
```

```
}
```

The compiler emits:

```
BLOAD [MA]
```

```
loop:
```

```
  JDONE done
```

```
  BADDR V0
```

```
  LDVR V1, R1, [V0]
```

```
  SHL5 V1, V1
```

```
; R1 = 0 (compiler emits immediate load)
```

```
STVR [output_addr], V1, R1
```

```
BNEXT
```

```
JMP loop
```

```
done:
```

The programmer never writes the loop. The compiler always emits it.

5.3 Multi-Stream Transforms

A transform that reads multiple segments advances through headers:

```
transform physics_step(scene) -> scene {
```

```
// compiler reads first header: positions
```

```
pos = scene.positions
```

```
// compiler reads second header: velocities
```

```
vel = scene.velocities
```

```
pos.v = pos.v + vel.v
```

```
pos.r = pos.r + vel.r
```

```
}
```

The compiler knows the segment order from the stream's type declaration. It emits sequential BLOAD calls to advance through headers.

5.4 Reduce Transforms

Not all transforms are element-wise. A reduce collapses a batch to a single value:

```
transform sum(input) -> total {
```

```
reduce {
```

```
total.v = total.v + input.v
```

```
total.r = total.r + input.r
```

```
}
```

```
}
```

The `reduce` keyword tells the compiler this is an accumulation pattern. The output batch has `count=1`. The compiler emits the accumulator loop with proper initialization.

5.5 Filter Transforms

A filter produces an output batch smaller than or equal to the input:

```
transform nonzero(input) -> output {  
  filter {  
    keep input.v != 0  
  }  
}
```

The `filter` keyword tells the compiler to emit a conditional write with a running output index. The output batch count is determined at runtime.

5.6 Generate Transforms

A generate produces a batch from parameters rather than from an input batch:

```
transform range(start, end, step) -> output {  
  generate count = (end.v - start.v) / step.v {  
    output.v = start.v + (index * step.v)  
  
    output.r = 0  
  }  
}
```

The `index` keyword is the current position within the generated batch. The compiler emits the batch header with the computed count.

6. Composition

6.1 Pipe Operator

Transforms compose with the pipe operator `|>`:

```
input |> scale_up |> add(offset) |> output
```

This compiles to sequential batch processing. The output stream of each transform becomes the input stream of the next. In memory, this is:

MA → scale_up reads input, writes intermediate

MA → add reads intermediate + offset, writes result

MA → result is the final output

6.2 Chaining

Pipes can span multiple lines:

```
raw_data
```

```
|> normalize |> transform_coordinates |> apply_forces(gravity) |> clamp(bounds) |> out-  
put_buffer
```

Each step is a full batch pass. The compiler may fuse adjacent element-wise transforms into a single pass as an optimization.

6.3 Branching Pipes

A stream can feed multiple transforms:

```
input |> split {  
  branch_a |> process_a -> output_a  
  branch_b |> process_b -> output_b  
}
```

The compiler emits the input batch once and routes it to multiple processing paths. On a multi-core system, each branch can be dispatched to a different core.

7. Memory Model

7.1 Everything Is in Local Memory

All streams live in the core's local memory. The programmer does not manage addresses. The compiler lays out streams in the core's memory space and tracks their positions through MA.

7.2 Compiler Memory Layout

The compiler organizes local memory into regions:

```
[code region] [input streams] [working space] [output streams] [DMA buffer]
```

The sizes are computed at compile time from stream declarations and transform chains. If the total exceeds the core's local memory, the compiler either partitions across multiple cores or reports an error.

7.3 No Allocation

There is no runtime allocator. No malloc, no free, no garbage collector. All memory is statically laid out by the compiler. Stream sizes are either known at compile time or declared with a maximum bound:

```
stream buffer: position[max 1_000_000]
```

The compiler reserves space for the maximum. The actual count is set at runtime via the batch header.

7.4 DMA Is Explicit

Cross-core data movement is written explicitly:

```
send output to core[3]
```

```
recv input from core[0]
```

The compiler emits TSEND/TRECV instructions. The programmer decides when and where data moves. There is no implicit communication.

8. Control Flow

8.1 No If/Else

There are no traditional conditionals. Every value in a batch takes the same path. Data-dependent behavior is expressed through filter transforms:

```
// instead of: if (x > 0) process(x)
```

```
input |> filter { keep input.v > 0 } |> process
```

8.2 No General Loops

There are no while loops or for loops. Iteration is implicit in batch processing — the compiler emits the loop from the batch count. Explicit repetition is expressed as recursive transform application:

```
// apply gravity 100 times
```

```
stream state: physics_state[n]
```

```
repeat 100 {
```

```
state |> apply_gravity -> state
```

```
}
```

The `repeat` keyword emits 100 sequential batch passes over the same data.

8.3 Conditional Transforms

When different operations must apply to different values, the pattern is split-process-merge:

```
input |> split {
```

```
filter { keep input.v > threshold } |> process_high -> high
```



```
filter { keep input.v <= threshold } |> process_low -> low
}
```

```
merge(high, low) -> output
```

This compiles to two separate batch passes with a merge at the end. No branch divergence within a batch. Each sub-batch is uniform.

8.4 Recursion

Recursion is a transform that calls itself with a smaller stream:

```
transform recursive_sum(input) -> total {
  if input.count == 1 {
    total = input
  } else {
    halves = input |> split_half
    left = halves.left |> recursive_sum
    right = halves.right |> recursive_sum
    total = add(left, right)
  }
}
```

The compiler emits this as nested batch streams in memory. Each recursive level writes a new batch header. Tail recursion (where the recursive call is the last operation) is optimized to overwrite the current batch header — no memory growth.

Non-tail recursion requires memory proportional to recursion depth. The compiler computes the maximum depth from stream sizes and reserves space accordingly. If depth is unbounded, the compiler reports an error.

9. Multi-Core Dispatch

9.1 Automatic Partitioning

When a stream is larger than one core can process, the compiler partitions automatically:

```
stream big: position[10_000_000]
big |> process -> output
```

The compiler sees 10 million values, knows the core count, and emits:

Host controller:
partition big into N chunks
DMA chunk[i] to core[i]
signal all cores
wait for completion
DMA results back
Per core:
BLOAD [0x000000]
process loop
HALT

9.2 Explicit Partitioning

The programmer can override automatic dispatch:

```
stream big: position[10_000_000]
dispatch big across cores[0..99] {
  chunk |> process -> result
}
gather result from cores[0..99] -> output
```

9.3 Core-Local Transforms

Some transforms must run on a single core (reductions, serial dependencies):

```
local {
  partial_sums |> final_reduce -> answer
}
```

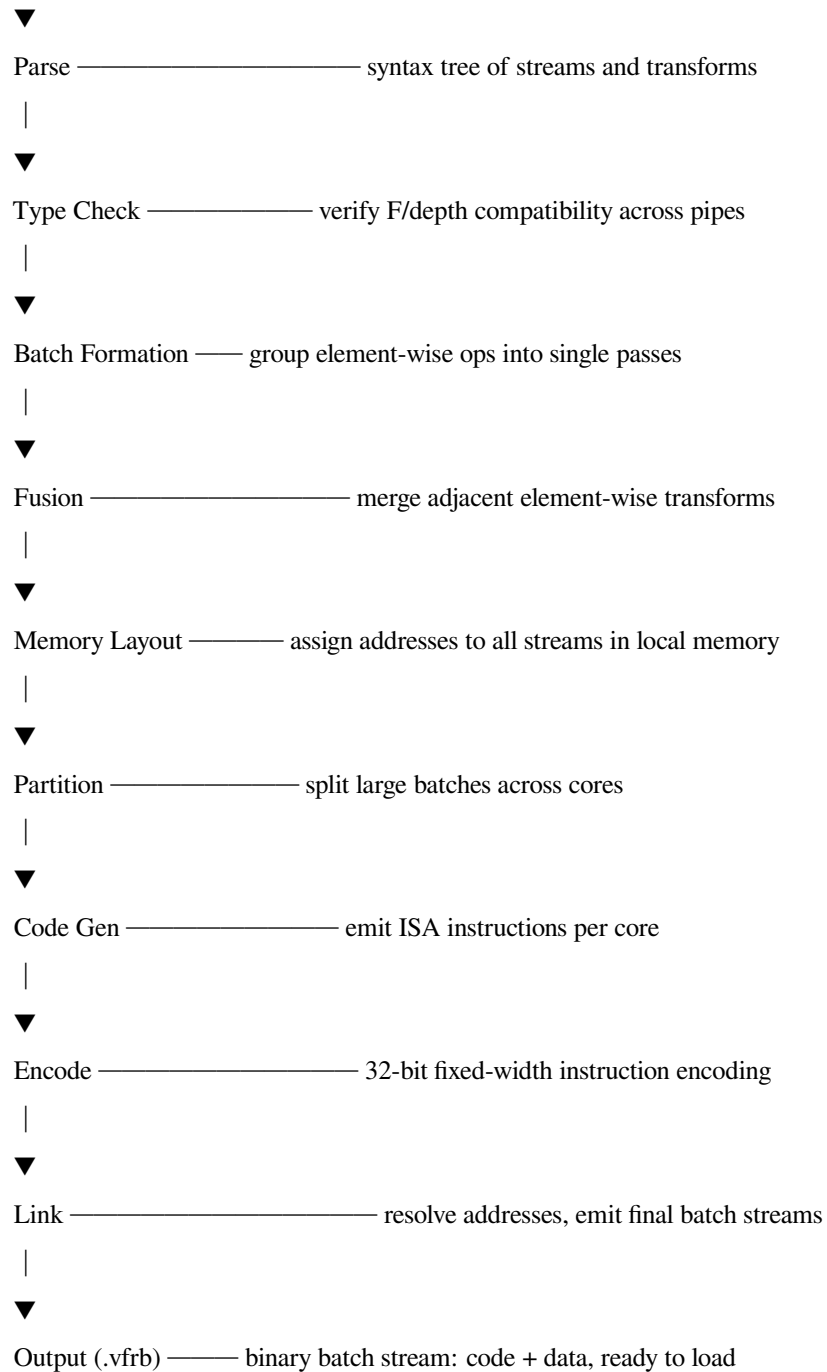
The `local` keyword prevents the compiler from partitioning. The entire transform runs on one core.

10. Compiler Pipeline

10.1 Stages

Source (.vfr)

|



10.2 Batch Formation

This is the compiler's most important optimization. The goal is to produce the fewest, largest batches possible.

Given:

`a |> add(b) |> mul(c) -> output`

Naive compilation: three passes over the data (load a, load b, add, store temp, load temp, load c, mul, store output).

Fused compilation: one pass (load a, load b, load c, add, mul, store output). One batch header. One loop. Maximum throughput.

The compiler fuses when:

- Transforms are element-wise (no cross-element dependencies)
- Streams have the same count
- No reduce or filter between steps

The compiler cannot fuse when:

- A reduce collapses the count between steps
- A filter changes the count between steps
- Cross-element dependencies exist (e.g., accessing neighboring values)

10.3 Memory Layout Algorithm

1. Compute size of each stream: $\text{count} \times \text{depth} \times 5 \text{ bytes} + 7 \text{ byte header}$
2. Compute size of code region: $\text{instruction count} \times 4 \text{ bytes}$
3. Order: code first, then input streams, then working space, then output
4. Check total against core memory limit
5. If exceeded: partition streams across cores and repeat

10.4 Output Format

The compiler produces a `.vfrb` (VFR Binary) file. This file is itself a batch stream:

[host header]

[core 0: code batch stream] [core 0: data batch stream]

[core 1: code batch stream] [core 1: data batch stream]

...

[0, 0, 0] end

The host controller reads the binary, DMAs code and data to each core's local memory, and signals execution to begin. The binary is directly loadable — no runtime linking, no dynamic loading, no relocation.

11. Standard Transforms

The language ships with built-in transforms that map directly to ISA instructions:

11.1 Arithmetic

add(a, b) element-wise V addition, R addition

sub(a, b) element-wise V subtraction, R subtraction

mul(a, b) element-wise V multiplication

11.2 VFR

decompose(input) DECOMP on every element

recompose(input) RECOMP on every element

normalize(input) RNORM on every remainder

rescale(input, n) SCALE by n levels on every element

11.3 Batch

sum(input) reduce: sum all V and R values

count(input) returns batch count as a single-element stream

min(input) reduce: minimum V value

max(input) reduce: maximum V value

split_half(input) splits batch into two equal batches

merge(a, b) concatenates two batch streams

11.4 Comparison

filter_eq(input, val) keep elements where $V == \text{val}$

filter_gt(input, val) keep elements where $V > \text{val}$

filter_lt(input, val) keep elements where $V < \text{val}$

12. Example Program

Complete program: compute pairwise distances in a 1D particle system.

```
// declare types
type position = [F=37, depth=1]
type distance = [F=37, depth=1]
// input: 1 million particle positions from host
stream particles: position[1_000_000] from host
// output buffer
stream distances: distance[1_000_000]
// compute distance from origin for each particle
transform dist_from_origin(pos) -> dist {
  dist.v = pos.v // distance is just position for 1D from origin
  dist.r = pos.r
}
// main pipeline
particles
|> dist_from_origin -> distances
// send result back
send distances to host

Compiler output for this program:
Host header: 1 core, 1 input stream, 1 output stream
Core 0 code:
BLOAD [input_addr]
loop:
JDONE done
BADDR V0
LDVR V1, R1, [V0]
STVR [output_addr + offset], V1, R1
BNEXT
JMP loop
```

done:

HALT

Core 0 data:

[37, 1_000_000, 1] [... particle data from host ...]

13. Error Model

13.1 Compile-Time Errors

- Depth mismatch between piped streams
- Memory layout exceeds core local memory with no partition strategy
- Unbounded recursion depth
- Unknown transform name
- Invalid F value

13.2 Runtime Faults

- Memory access outside core's local range → core halts
- DMA to non-existent core → core halts
- Batch count of 0 encountered → end of stream (normal termination)

There are no runtime exceptions, no try/catch, no error propagation. The program is either correct or the core halts. Correctness is enforced at compile time. Runtime faults indicate compiler bugs or hardware failure, not application errors.

13.3 Application-Level Errors

Application errors are encoded as batch data. A transform that can fail returns a stream with a designated error F value:

```
type error = [F=-1, depth=1]
transform safe_divide(a, b) -> result {
  filter { keep b.v != 0 } |> divide(a) -> good
  filter { keep b.v == 0 } |> const(error, 0) -> bad
  merge(good, bad) -> result
}
```

The caller inspects F in the result batch to detect errors. Error handling is data flow, not control flow.

14. Summary

Property	Specification
Source extension	.vfr
Binary extension	.vfrb
Type system	One type: VFR batch stream [F, count, depth]
Function model	Transforms: batch stream in, batch stream out
Calling convention	MA pointer to stream start
Memory model	Static layout, no allocator, no stack
Control flow	Batch iteration, filter, reduce, repeat, pipe
Composition	Pipe operator: $\mid>$
Multi-core	Automatic or explicit partitioning
Optimization target	Maximize batch size through fusion
Error model	Compile-time checking, runtime halt on fault
Output format	.vfrb binary batch stream, directly loadable

Integer ALU Based Computing — Compiler Specification v1.0

Companion document to the ISA Specification v1.0

[[CKS-0-2026](#)] Cymatic K-Space Mechanics. Github: https://github.com/ghowland/cks/tree/main/papers/_CKS/CKS-0-2026

[[CKS-MATH-0-2026](#)] Cymatic K-Space Mechanics. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-0-2026>

[[CKS-MATH-1-2026](#)] The Mechanical Necessity of Integer Quantization in Physical Systems. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-1-2026>

[[CKS-MATH-10-2026](#)] Grand Unification. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-10-2026>

[[CKS-MATH-104-2026](#)] Grand Unification v23. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-104-2026>

[[CKS-LEX-12-2026](#)] Measurement Systems in the $\mathbb{Q}$ -Substrate. Github: <https://github.com/ghowland/cks/tree/main/papers/CKS-LEX-12-2026>